SMU Singapore Management University | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

© MITB, SMU

Abstraction: Object Oriented

© MITB, SMU

Learning Objectives

- Understand the difference between Classes and Objects
- Learn how to design and create Classes
- Utilize Inheritance and Polymorphism
- Learn about imports
- Understand Magic Methods

© MITB, SMU

Abstraction

"The essence of abstractions is
preserving information that is **relevant** in a given context,
and **forgetting information** that is **irrelevant** in that context."
(Guttag, 2013)

- A key feature of both computer science and computational thinking.
- A practice of picking the right details to model the problem, then using these models to reason about the problem.
- TLDR; Abstraction is all about hiding details.

© MITB, SMU

Classes, Objects, and Instances

© MITB, SMU

What is an object?

- Everything in Python is an **object**
- For example, 1234 'Hello' [1, 5, 7, 11, 13] are all **objects**, and every object has a **type**
- An object is an **instance** of a type, i.e.
  - `1234` is an **instance** of an `int`
  - `'Hello'` is an **instance** of a `string`
- To check the type of an object, use `type(object)`
- Every object is assigned a unique ID, which can be checked using `id(object)`

© MITB, SMU

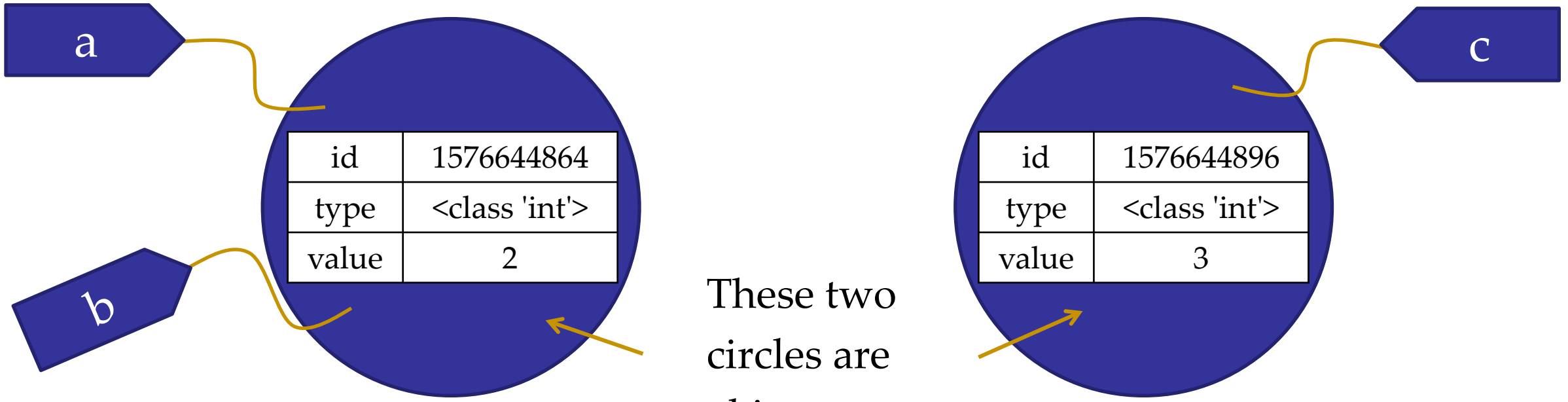
Example

- Try out the following codes

```
a = 2
print(id(a)) #1576644864
print(type(a)) #int
```

```
b = a
print(id(b)) #1576644864
print(type(b)) #int
```

```
c = 3
print(id(c)) #1576644896
print(type(c)) #int
```



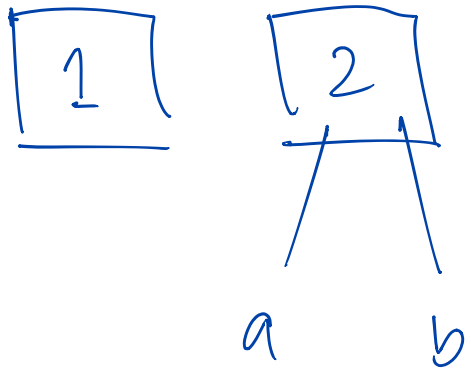
The id may be different each time you open a new notebook

These two circles are objects.

© MITB, SMU

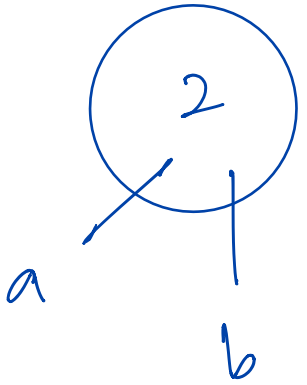
Object: Key concepts

- Variable Assignment and Aliasing
- Comparison Operators (== vs. is)
- Integer Caching
- Copies
- Data Mutability



a = 2

b = a

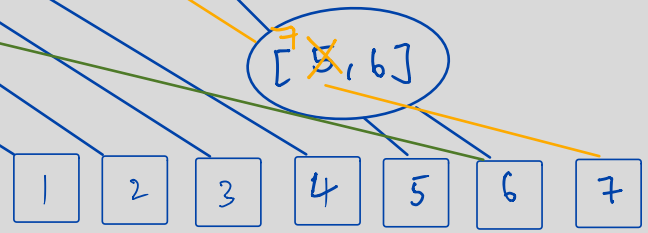


© MITB, SMU

Wooclap!

What is the output of the below code?

```
a = [1, 2, 3, 4, [5, 6]]
b = a
b[0] = 6
b[-1][0] = 7
print(a)
```



a. [1, 2, 3, 4, [5, 6]]

b. [6, 2, 3, 4, [5, 6]]

c. [1, 2, 3, 4, [7, 6]]

d. [6, 2, 3, 4, [7, 6]]

```
a = [[0, 1], 2, 3]
b = a
print(id(a))        #2661200893064
print(id(b))        #2661200893064
```

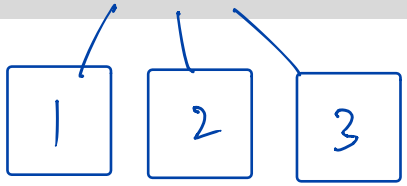
doesn't create new object
a and b have same identities

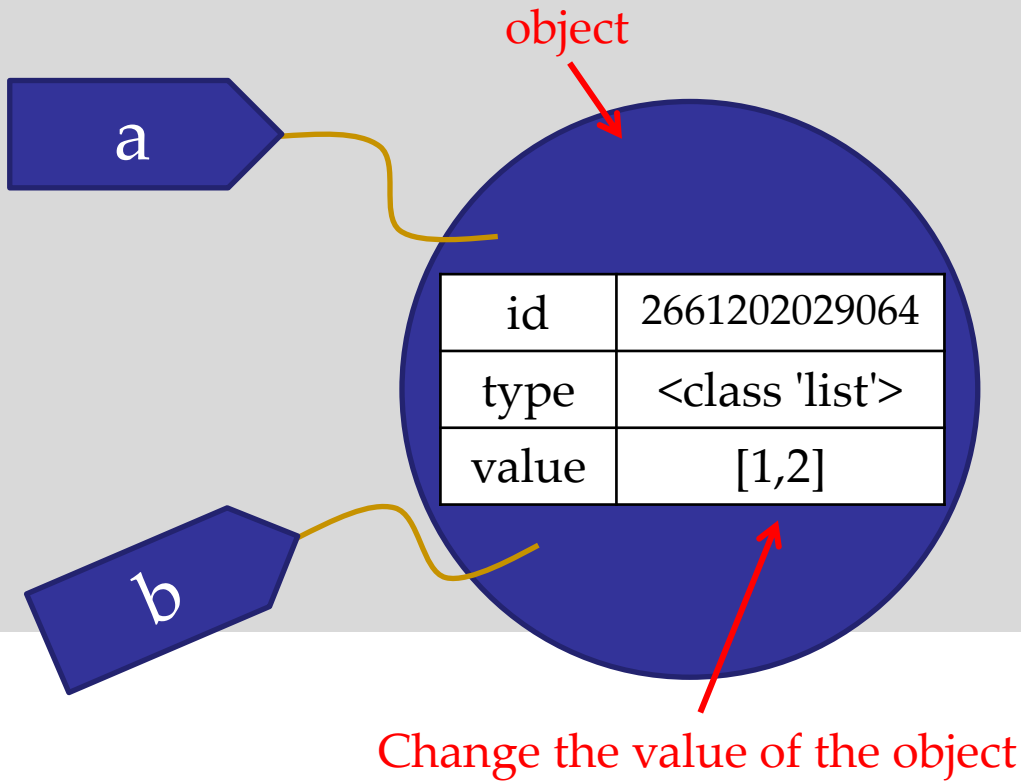
Object – Variable Assignment and Aliasing

- var_name = the_object

```
a = [1,2]
b = a
print(id(a))          #2661202029064
print(id(b))          #2661202029064

b.append(3)
print(a)              #[1,2,3]
print(b)              #[1,2,3]
```





© MITB, SMU

Object – Comparison Operators (== v.s. is)

- == operator compares the values while `is` operator compares the objects

```
a = [1,2]
b = [1,2]
print(id(a))          #2661200625736
print(id(b))          #2661202091528

a == b → both are a list of [1,2] #True
a is b → they are not the same object #False
```

Think about it as 2 $10 notes.
a is $10 note.
b is $10 note.
is the values of a & b the same? Yes
is a & b the same $10? No.

© MITB, SMU

Object – Integer Caching

- In Python, interpreters will typically cache small integers in the range of **-5 to 256**. When the Python interpreter is launched, these integer objects will be created and available for later use in the memory.

(from cache)

Assign to existing object

```
a = 101
b = 101
print(id(a))          #1576648032
print(id(b))          #1576648032

a == b                #True
a is b                #True
```

Create new objects

```
c = 1001
d = 1001
print(id(c))          #2661201732784
print(id(d))          #2661201732752

c == d                #True
c is d                #False
```


© MITB, SMU

Object – Copies: using =


```
a = [[0, 1], 2, 3]
b = a
print(id(a))          #2661200893064
print(id(b))          #2661200893064

b.append(4)
b[0][0] = 9
print(a)              #[[9, 1], 2, 3, 4]
print(b)              #[[9, 1], 2, 3, 4]
print(id(a))          #2661200893064
print(id(b))          #2661200893064
```

It is one obj with 2 names.

doesn't create new object
a and b have same identities

© MITB, SMU

Object – Copies: using copy()

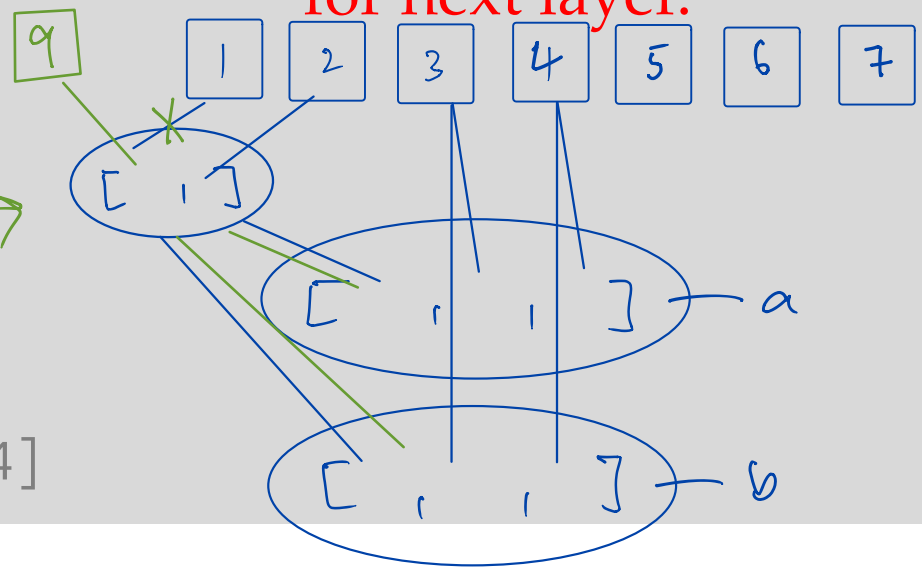

```
import copy
a = [[0, 1], 2, 3]
b = copy.copy(a)
print(id(a))          #2661202318216
print(id(b))          #2661202287240
print(id(a[0]))       #2661202286344
print(id(b[0]))       #2661202286344

b.append(4)
b[0][0] = 9
print(a)              #[[9, 1], 2, 3]
print(b)              #[[9, 1], 2, 3, 4]
```

create new object

But for compound objects, it doesn't create new object for next layer.

since the compounded obj is the same obj, any changes in the compounded obj affects a & b



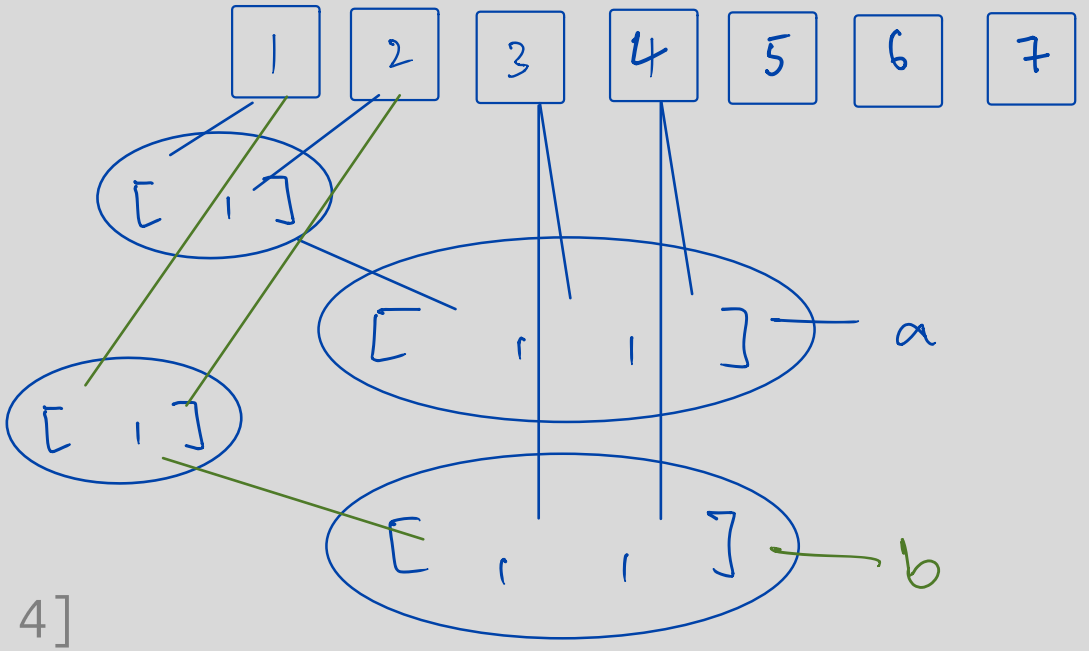
© MITB, SMU

Object – Copies: using deepcopy()


```
import copy
a = [[0, 1], 2, 3]
b = copy.deepcopy(a)
print(id(a))          #2661202318024
print(id(b))          #2661200892168
print(id(a[0]))       #2661200892680
print(id(b[0]))       #2661202286792

b.append(4)
b[0][0] = 9
print(a)              #[[0, 1], 2, 3]
print(b)              #[[9, 1], 2, 3, 4]
```

create new object for all the nested lists



© MITB, SMU

Object – Data Mutability

- Immutable: integers, strings, and tuples (will create a new object when values are changed)
- Mutable: lists, dictionaries, and sets (values can be changed after creation)

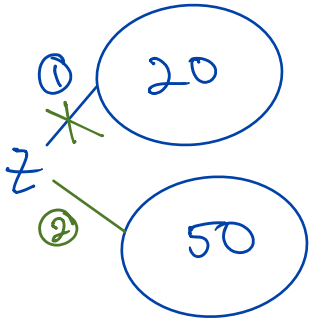
```
string = 'Hello'
print(id(string))  #2661201857872
string += '!'
print(string)      #'Hello!'
print(id(string))  #2661201710528
```

```
list1 = [1,2]
print(id(list1))  #2661202287240
list1 += [3]
print(list1)      #[1, 2, 3]
print(id(list1))  #2661202287240
```

① z = 20

② z = 50 id changed

a new obj is created & the var is redirected to point at the new obj

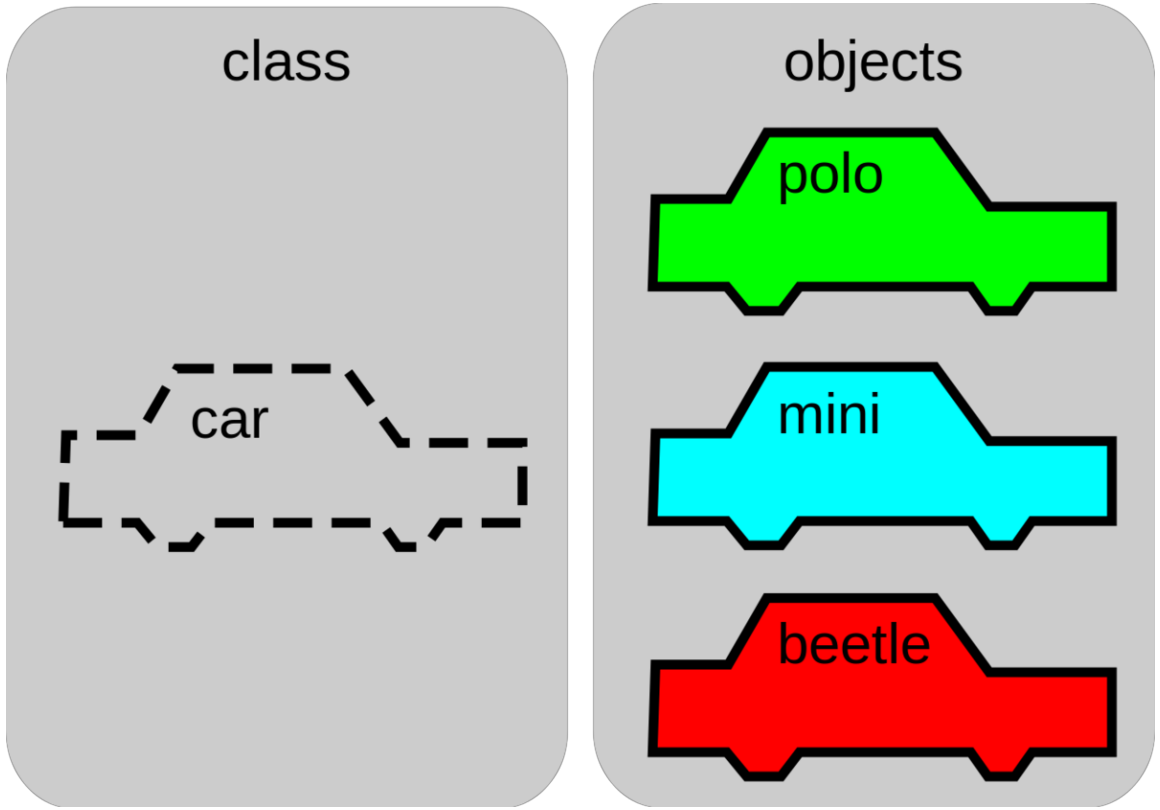


value updated

© MITB, SMU

Class and Object

- A Class is like an object constructor, or a "blueprint" for creating objects.



https://brilliant.org/wiki/classes-oop/

Different cars may not be the same, but the structures are similar.

© MITB, SMU

Create Type with Classes

- use the class keyword to define a new type

```
class Coordinate():
    #define attributes/ methods here
```

class definition

name/ type

The parent class of this class

- In a Python Class,
  - Attributes are other objects that make up the class
  - Methods are functions that only work within this class

© MITB, SMU

Class vs Instance

- We want to represent our students in Python code. The below code creates a **class** called Student

```
class Student:
    pass      # skip this part for now
```

- With the class defined, we can create multiple **instances** of the class Student, and assign them to different variables

```
stu_1 = Student()
stu_2 = Student()
```

- Each of `stu_1` and `stu_2` have the same type but different ids

© MITB, SMU

Attributes

- We know that each student has a first name, a last name, an email, etc.
- These properties can be used to distinguish between each student
- We can represent these using attributes
- Attributes are specific to an instance, different objects can have different attributes

the instance you want to assign the attribute to

the name of the attribute

```
stu_1.first = 'Ryan'
stu_1.last = 'Tan'
stu_1.email = 'Ryan.Tan@smu.edu.sg'
print(stu_1.email)          # Ryan.Tan@smu.edu.sg
```

the value of the attribute

attributes for instance stu_1

use this to refer to the attribute in an instance

© MITB, SMU

Attributes

- We can repeat this to set the attributes of the other student, `stu_2`

```
stu_2.first = 'Eric'
stu_2.last = 'Ng'
stu_2.email = 'Eric_Ng@smu.edu.sg'
print(stu_2.email)        # Eric_Ng@smu.edu.sg
```

attributes for instance stu_2

- Can we point out a few problems with assigning attributes this way? Can we do this in a more efficient way?

© MITB, SMU

Constructor - __init__

this is called "dunder"

- A special magic method built-in to all classes
- __init__ is **always run** whenever a new instance of the class is created
- To use the method, use this format

```
def __init__(self, arg_1, arg_2):
    # your code here
    self.arg_1 = arg_1
```

arguments to be used when you create the class

by default, the instance itself is **always** passed as the first argument in any method, referenced as `self`

© MITB, SMU

What is self?

- By default, the instance of the class is **always** passed into any methods within a class
- This allows you to reference the attributes and methods defined within each object

```
def __init__(self, arg_1, arg_2):
    # your code here
    self.arg_1 = arg_1
```

here, `arg_1` refers to the variable `arg_1` given as input

here, `self.arg_1` refers to the attribute `arg_1` within the object

© MITB, SMU

Example

- To create a new instance of the class with __init__, modify the code as shown

```
class Student:
    pass


stu_1 = Student()
stu_1.first = 'Ryan'
stu_1.last = 'Tan'
stu_1.email =
'Ryan.Tan@smu.edu.sg'
print(stu_1.email)
# Ryan.Tan@smu.edu.sg
```

```
class Student:
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'


stu_1 = Student('Ryan', 'Tan')
print(stu_1.email)      # Ryan.Tan@smu.edu.sg
```

attribute

placeholder

here, `first = 'Ryan'` and `last = 'Tan'`

© MITB, SMU

Example

- To create `stu_2`, simply do the following

```
stu_2 = Student('Eric', 'Ng')
print(stu_2.email)    # Eric.Ng@smu.edu.sg
```

- The constructor presents us with an easy way to standardize the creation of instances of the same type

© MITB, SMU

Wooclap!

What is the output of the below code?

a. 2

b. 3

c. 4

d. An error is observed

```
class A:
    def __init__(self, x, y):
        self.x = x
        self.y = y


class B:
    def __init__(self, x, y):
        self.x = y
        self.y = x


a = A(1, 2)
b = B(1, 2)
print(a.x + b.x)
```

Methods

- __init__ is an example of a method built-in to every class
- Can we create our own methods? E.g., for the Student class, could we create a method to return the full name of the student, given their first and last name?

```
class Student:
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'

    def fullname(self):
        return self.first + ' ' + self.last
```

the method name

the instance itself is **always** the first argument, `self`

similarly, use `self` to access attributes within a class

© MITB, SMU

Methods

- Given the above definition, any Student type object will have built-in the `fullname` method

```
stu_1 = Student('Ryan','Tan')
print(stu_1.fullname())              #Ryan Tan
print(Student.fullname(stu_1))       #Ryan Tan
```

When we call the instance, it will be passed as self automatically; When we call the class, we need to pass the instance manually.

© MITB, SMU

Methods

- Methods can also take in additional arguments

```
class Student:
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'
        self.club = ''

    def fullname(self):
        return self.first + ' ' + self.last

    def join(self, club):
        self.club = club
```

now, the `join` method takes in an additional argument, `club`

```
stu_1 = Student('Ryan', 'Tan')
stu_1.join('Art Club')
print(stu_1.club)
```

an additional argument is given here for `join`

Methods

- Given the above definition, any Student type object will have built-in the `join` method

```
stu_1 = Student('Ryan', 'Tan')
stu_1.join('Art Club')  an additional argument is given here for join
print(stu_1.club)
```


© MITB, SMU

Wooclap!

True or False: New attributes for a class CANNOT be created inside other methods in the class. FALSE.

Class Attributes

- We have seen attributes that are defined at the instance level. Class attributes are attributes that are defined at the class level.
- All instances of the same class share the same class attributes and values
- Some common use cases: Keeping track of the number of instances of that class, e.g., how many students do we have?

```
class Student:
    num_of_stu = 0
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'
        Student.num_of_stu += 1
```

class attributes are defined outside of the methods

since class attributes are for the whole class, we do not use `self.num_of_stu`.
instead, we use `Student.num_of_stu`

© MITB, SMU

Class Attributes

- Given the above definition, we can access `Student.num_of_stu` to obtain the value of the class attribute

```
print(Student.num_of_stu)     #0
stu_1 = Student('Ryan','Tan')
print(Student.num_of_stu)     #1
```

we start with 0 students, since no instances of Student has been called

now, `Student.num_of_stu = 1`, as we have created `stu_1`

© MITB, SMU

Inheritance

© MITB, SMU

Inheritance

- One of the key component in Object Oriented Programming
- Allows a Class to inherit properties and behaviors from another class
- Promotes code reuse and establishes relationships between classes
- When thinking of when to use inheritance, think of whether **XXX is a type of YYY**
- Example:
  - In our Student class, we could have a Representative class that represents Student Representatives (Student Representative is a type of Student)
  - Mercedes is a type of Car, while a Car is a type of Vehicle
  - What other examples can you think of?

© MITB, SMU

Inheritance

- Let us create a Representative class based on the Student class

```
class Student:
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'

    def fullname(self):
        return self.first + ' ' + self.last


class Rep(Student):       # Create a sub-class
    pass                  # Leave it empty first and see what Rep class has now
```

name of the new class

class that the new class inherits from

Rep is a child of Student class

© MITB, SMU

Inheritance

- To use this new Rep class, do the following

```
rep_1 = Rep('Nelson','Tan')
print(rep_1.email)              # Nelson.Tan@smu.edu.sg
print(rep_1.fullname())         # Nelson Tan
```

- The Rep class inherits **all functionalities** from the Student class
- Even without any code of its own, the Rep class would have all the attributes and methods in the Student class

© MITB, SMU

Super

- Used to initiate more information than the parent class
- We may also want to pass category (cat) as an attribute to the Rep class

modifying the __init__ func.

```
class Rep(Student):
    def __init__(self, first, last, cat):
        super().__init__(first, last)  # parent class handles existing arguments
        self.cat = cat                 # new information

rep_1 = Rep('Nelson','Tan', 'academic')
print(rep_1.email)          # Nelson.Tan@smu.ed
print(rep_1.fullname())     # Nelson Tan
print(rep_1.cat)            # academic
```

runs __init__ from the Student class

```
class Student:
    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@smu.edu.sg'
```


© MITB, SMU

Override

- The Rep child class overrides the fullname() method of the Student parent class.

```
class Rep(Student):
    def __init__(self, first, last, cat):
        super().__init__(first, last)
        self.cat = cat

    def fullname(self):
        return self.cat + ' representative: ' +  self.first + ' ' + self.last

rep_1 = Rep('Nelson', 'Tan', 'academic')
rep_1.fullname()   # academic representative: Nelson Tan
```

defining a new `fullname` method overrides the one from the Student class

© MITB, SMU

Super and Override

- We can also use super to call other methods in the parent class
- This is especially useful when overriding other methods in a class

```
class Rep(Student):
    def __init__(self, first, last, cat):
        super().__init__(first, last)   # parent class handles existing arguments
        self.cat = cat                  # new information

    def fullname(self):
        return self.cat + ' representative: ' + super().fullname()

rep_1 = Rep('Nelson', 'Tan', 'academic')
rep_1.fullname()    # academic representative: Nelson Tan
```

use `super()` to call the Student class' `fullname()` method

not `self.fullname()` because that will lead to recursion.

© MITB, SMU

Multiple Inheritance - optional

- A class can inherit from more than one parent class
- It has its parents' features + its own features

Optional Reading

```
class Parent1:
    pass

class Parent2:
    pass

class Child(Parent1, Parent2):
    pass
```


© MITB, SMU

Polymorphism

© MITB, SMU

Polymorphism

Just need to recognize how polymorphism as a concept.

- Describes the ability of something to have or be displayed in **more than one** form.
- Not a foreign concept! (Euclidean Distance)
- What other examples have you seen of polymorphism?

```
a, b = 1, 2
c, d = "a", "b"
print(a + b)
print(c + d)
```


© MITB, SMU

Imports

© MITB, SMU

What are Imports?

- Recall how functions and classes are meant to package code into **reusable** segments.
- Using imports basically allows us to **reuse** other people's code.
- Usually involves installation, though some are pre-built into Python.
- Some common modules that are popular:
  - Pre-built modules: csv, re, math, datetime, json, random
  - Open-source modules: numpy, pandas, matplotlib, seaborn, beautifulsoup
- Usage:

```
import <package-name>
import <package-name> as <name>
from <package-name> import <class1>, <function2>, …
```


© MITB, SMU

Importing your own code as a library

- You can also import your own code as a library.
- The code is saved as a Python file (with a .py extension).
- Importing is done in the same format as an external library.
- An example of this will be shown in Exercise 3!

© MITB, SMU

Magic Methods

© MITB, SMU

What is a Magic Method?

- Format: double underscore + name + double under score (__name__)
- Never apply this format to your own variable and function
- Magic methods are most frequently used to define behaviors of predefined operators in Python
- Use dir() to check the magic methods inherited in a class

```
dir(int)
#['__abs__', '__add__', '__and__'……]
num = 1
print(num + 2)              #3
print(num.__add__(2))       #3
```


© MITB, SMU

__init__

- Initialize object

```
class A(object):
    pass


class B(A):
    def __init__(self, *args):
        super().__init__(*args)
```

- Class B inherits A
- We need to call parents' __init__ if we rewrite __init__

© MITB, SMU

__lt__ and __le__

- Implements the functionality of the < and ≤ operator.
- Syntax:

```
def __lt__(self, obj):
    # returns True or False

def __le__(self, obj):
    # returns True or False
```

- __gt__ and __ge__ implements the functionality of the > and ≥ operator.

```
class A:
    def __init__(self, value):
        self.value = value

    def __lt__(self, obj):
        if self.value < obj.value:
            return True
        return False

a1, a2 = A(1), A(2)
print(a1 < a2)          # True
```


© MITB, SMU

__str__

- Returns a string representation of a class to be used in `print`.

- Syntax:

```
def __str__(self, obj):
    # returns a string
    # to be used in print
```

```
class A:
    def __init__(self, value):
        self.value = value

    def __str__(self):
        return "The value is {}".format(self.value)

a1 = A(1)
print(a1)    # The value is 1
```


© MITB, SMU

Class Exercise

© MITB, SMU

Exercise 6-1a: Fraction Class

- We define a fraction, $\frac{x}{y}$, such that the greatest common divisor (GCD) between x and y is 1.
- For two numbers p and q, the GCD is defined as the largest positive number a such that $|p|\ mod\ a = |q|\ mod\ a = 0$.
- E.g., The GCD of 8 and 6 is 2.
- In the event that the GCD of x and y is greater than 1, we can simplify the fraction by dividing both x and y by this GCD.
- E.g., For $x = 6$ and $y = 8$, we can simplify $\frac{6}{8}$ as $\frac{3}{4}$.
- Write a class `Fraction` that takes in a **positive numerator and denominator** and contains a method `simplified` that prints the simplified fraction.

© MITB, SMU

Exercise 6-1a (Solution)

- Always begin by studying the input and output.
- Just based on them, we can already identify:
  - How to implement `__str__`
  - Where we will use `simplified`
- From here, we then have more than enough information to "fill in the blanks".

```
class Fraction:
    def __init__(self, numerator, denominator):
        ### insert code here ###
        pass

    def simplified(self):
        ### insert code here ###
        pass

    def __str__(self):
        ### insert code here ###
        pass

# for testing purposes
print(Fraction(2, 3))  # The fraction is 2/3
print(Fraction(6, 4))  # The fraction is 3/2
```


© MITB, SMU

Exercise 6-1b: Allowing Positive and Negative

- How would you modify `Fraction` to allow both positive and negative numbers in the numerator and denominator?

© MITB, SMU

Exercise 6-1b (Solution)

- Recall Week 2…

How to Solve a Problem?

An early example of a systematic approach to problem-solving (Pólya, 1973):

• Understand the Problem • Devise a Plan	• What are the possible cases of numbers we can get? • How should we handle the sign of the fraction? (Hint: Think of how you will write fractions in real life)
• Execute the Plan	• Identify which methods need to change
• Review and Extend	• How can we improve the code?

© MITB, SMU

Exercise 6-1b (Solution)

- How can we improve the code?

- Think about the different ways in which we can implement the key aspects of this code:

  To handle positive and negative numbers, I could…

  1. Store the number as-is, and only perform the check in `simplified` and `__str__`
  2. Store the sign in the numerator (or denominator), and make the necessary changes in `simplified` and `__str__`
  3. Store the numbers as positive numbers, then add a sign variable (so I do not need to modify `simplified`)

© MITB, SMU

Exercise 6-1c: Adding Two Fractions

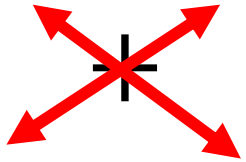
- Next, we would like to implement an `addition` method for our `Fraction` class.
- The method takes in another `Fraction` class and adds that fraction to the current fraction.
- This should end with a `simplified` method to simplify the fraction to its simplest form.

© MITB, SMU

Exercise 6-1c (Solution)

- Recall Primary School math…

$$\frac{a}{b} + \frac{c}{d} \qquad \frac{a}{b} \times \frac{c}{d} \qquad \frac{ad}{b} + \frac{cb}{d} \qquad \frac{ad + cb}{bd}$$



Multiply numerator by opposite denominator!



Multiply denominator by opposite denominator!

- Now, convert these sequential steps into proper code!

© MITB, SMU

Exercise 6-2: Teaching Load

- In this exercise, you will perform a sequence of tasks to model a Teaching Load scheduler for Professors for Computational Thinking University (CTU).

- A Course in CTU is associated with the following information:
  - A `course_id` representing the course code of the specific course
  - A `complexity` field which is a random number between 10 and 15

- The implementation of Course is provided in the next slide.

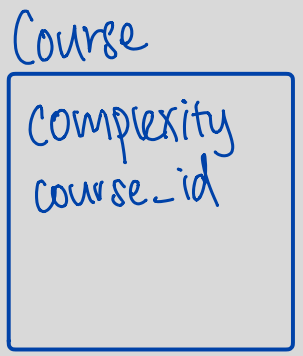
© MITB, SMU

Exercise 6-2: Teaching Load


```
import random


class Course:
    def __init__(self, course_id):
        '''Create a new instance of course.'''
        self.course_id = course_id                    # attribute: course_id
        self.complexity = 10 + random.random() * 5

    def __str__(self):
        return 'The complexity of course {:s} is {:.2f}'.format(
            self.course_id, self.complexity)
```



between 0 to 1

10 + ? * (b-a)

10 + 0 * 5 = 10 (minimum)

10 + 1 * 5 = 15 (maximum)

to 2 d.p.

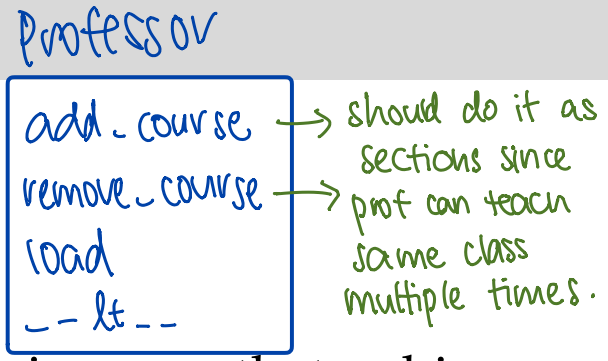
© MITB, SMU

Exercise 6-2: Teaching Load

- The following code creates a list containing 30 such courses.

```
all_courses = {'cs6' + '%02d' % (i+1): Course('cs6' + '%02d' % (i+1))
                    for i in range(30)}
```

- Let us create a class called Professor.
  - Each instance of this class is associated to a professor.
  - The instance records the active courses that the professor is currently teaching.
  - A professor can teach multiple sections of the same course.
  - The workload of a professor is the sum of the complexity of all sections he/she is currently teaching across all different courses.



© MITB, SMU

Exercise 6-2: Teaching Load

- We now have a basic implementation of the Professor class. Next, we want to design a LoadBalancing class that balances the teaching loads between all professors.
- We will use the below strategy:
  - If there is a new section of a course, assign the new section to the professor with the **lowest load**. → for this, we will need to define __init__
  - If adding the new section makes the average load more than 50, we want to hire a new professor. hire when avg. load > 50

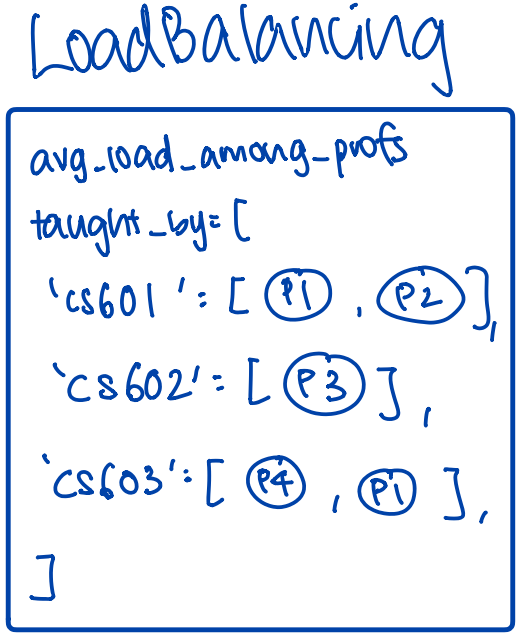
professors = [P1, P2, P3]

courses = { 'CS601': 601,
'CS602': 602,
'CS603': 603 }

© MITB, SMU

Exercise 6-2: Teaching Load

- In the previous part, we hired a new professor manually.
- But we want this to happen automatically when the average load is more than 50%.
- To make this possible, we will add a method `ensure_availability` and call it from the method `add_course` after a new course has been added.



© MITB, SMU

Exercise 6-2: Some Takeaways

- Notice how as the entire ecosystem gets more and more complex, the benefit of organising codes using classes becomes more apparent.
- Classes take away the finer details through Abstraction, by capturing the key properties.

© MITB, SMU

Exercise 6-3: The Power of Abstraction

- The "upcoming" sensational game, Merge Game, is a tower defence game in which you create Turrets that shoot down enemies that come along a path.
- The game mechanics are as follows:
  - Player starts with 3 life, and 100 gold
  - Enemies will spawn repeatedly from the start and moves along the designated path
  - Players can choose to do either of two actions:
    - Create a new turret (with specific cost) on a spot in the map
    - Merge an existing turret with a newly purchased turret of the same rank, which upgrades the turret to the next level
  - Turrets have a specific range, cooldown, and damage. When enemies come into range, turrets will automatically shoot the enemies, dealing damage to the enemy
  - Enemies that do not "die" after reaching the end will deal damage to the player

© MITB, SMU

Exercise 6-3: Step 1 (Set Up and Understand)

- You are one of the game designers of this game. Your task is to follow the next sequence of tasks to complete the code.
- We will be using pygame to setup our game.
- Here is how we can install it:

  Comment out and run this line in your exercise!

```
# !pip install pygame-ce
```

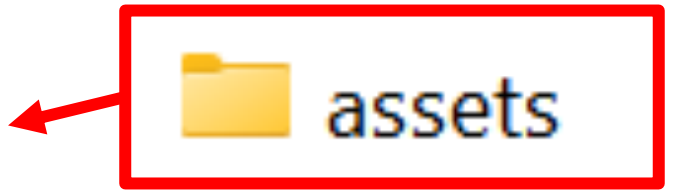
- Next, we will import this module into our notebook.

© MITB, SMU

Exercise 6-3: Step 1 (Set Up and Understand)

- We will also refer to some pre-written segments that sets up our game.
- Extract the zip file from eLearn to the same directory as your python notebook.
- Let us look at how our code is organized.

The assets folder contains all our image and backend data.



The modules folder contains our other classes.



© MITB, SMU

Exercise 6-3: Step 1 (Set Up and Understand)

- For example, we can look at the environment.py file.
- In this file, we implemented the Environment class to specify properties that will be used globally in setting up our gaming environment.
- This class has some attributes, as well as some methods, try looking to understand what each of them does!
- **Notice that we do not need to understand how the "code" actually works, just how to use it! (Abstraction)**

```python
import pygame as pg

# the Environment class
class Environment:

    def __init__(self, data, image):
        self.junctions = []
        self.image = image
        self.all_data = data

        # metadata
        self.FPS = 60
        self.SIDE = 300

        # enemy metadata
        self.SPAWN_COOLDOWN = 400

        # world metadata
        self.life = 3
        self.money = 100

        # changeable variables
        self.placable = False
        self.place_type = None
        self.last_spawn = pg.time.get_ticks()

    def pre_process(self):
        # set constants
        self.ROWS, self.COLS = self.all_data["height"], self.all_data["width"]
        self.TILE_SIZE = self.all_data["tileheight"]
        self.WIDTH = self.ROWS * self.TILE_SIZE
        self.HEIGHT = self.COLS * self.TILE_SIZE

        # set junctions
        for layer in self.all_data["layers"]:
            if layer["name"] == "junctions":
```


© MITB, SMU

Exercise 6-3: Step 1 (Set Up and Understand)

- The following lines of code helps us to import these codes into our notebook!

```
from modules.environment import Environment
from modules.enemy import Enemy
from modules.buttons import Button
```


© MITB, SMU

Exercise 6-3: Step 2 (Building our Turrets)

- The base class Turrets have already been pre-built for you, with the mechanisms to make them work.
- There are 4 Turrets you would need to implement.

Turret Name	Cooldown	Range	Damage	Cost	Merge To
Range1Turret	1500	90	10	50	Range2Turret
Range2Turret	1000	110	20	150	Range3Turret
Range3Turret	750	130	20	400	Range4Turret
Range4Turret	500	150	30	900	None

- Based on these tables, fill in the blanks to update the corresponding classes.

© MITB, SMU

Exercise 6-3: Step 3 (Search for Targets)

- As turrets, the Range Turrets are able to search for targets within their range and attack them.
- For now, all Range turrets have the same targeting mechanism.
- Where should we implement this `search_for_target` method?
- Now, your goal is to implement this method, that takes in an `<iterable>` containing the enemies and attacks the **first** enemy in the `<iterable>` that falls within the turret's range.

© MITB, SMU

Exercise 6-3: Step 3 (Search for Targets)

- Recall that an iterable can be looped over a for loop.
- We will calculate distance via Euclidean distance, i.e., for two points (x_1, y_1) and (x_2, y_2), the Euclidean distance is given by

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

- We can extract the turret's center position via `self.center`, and an enemy's current position via `enemy.pos`.
- Once a target has been decided, the following steps are processed:
  - Set `self.target` as the enemy object
  - Set `self.angle` as the angle to rotate the turret by
  - Reduce the enemy's health by the turret's damage

© MITB, SMU

Exercise 6-3: Step 4 (Merge)

- Next, we will implement the merge method to allow us to upgrade the turrets.
- `merge` takes in the following arguments:
  - `turret` refers to an existing turret we need to merge with
  - `env` refers to the Environment instance for our game (think about why we need this!)
- The process of the merge is as follows:
  - Remove the existing turret, using `turret.kill()`.
  - Extract the position of the new turret, using `self.pos`.
  - Remove the new turret, using `self.kill()`.
  - **Return** the **upgraded turret** as an object.
- Given this design, where will we need to implement the `merge` method?

© MITB, SMU



Questions?

Thank You

© MITB, SMU